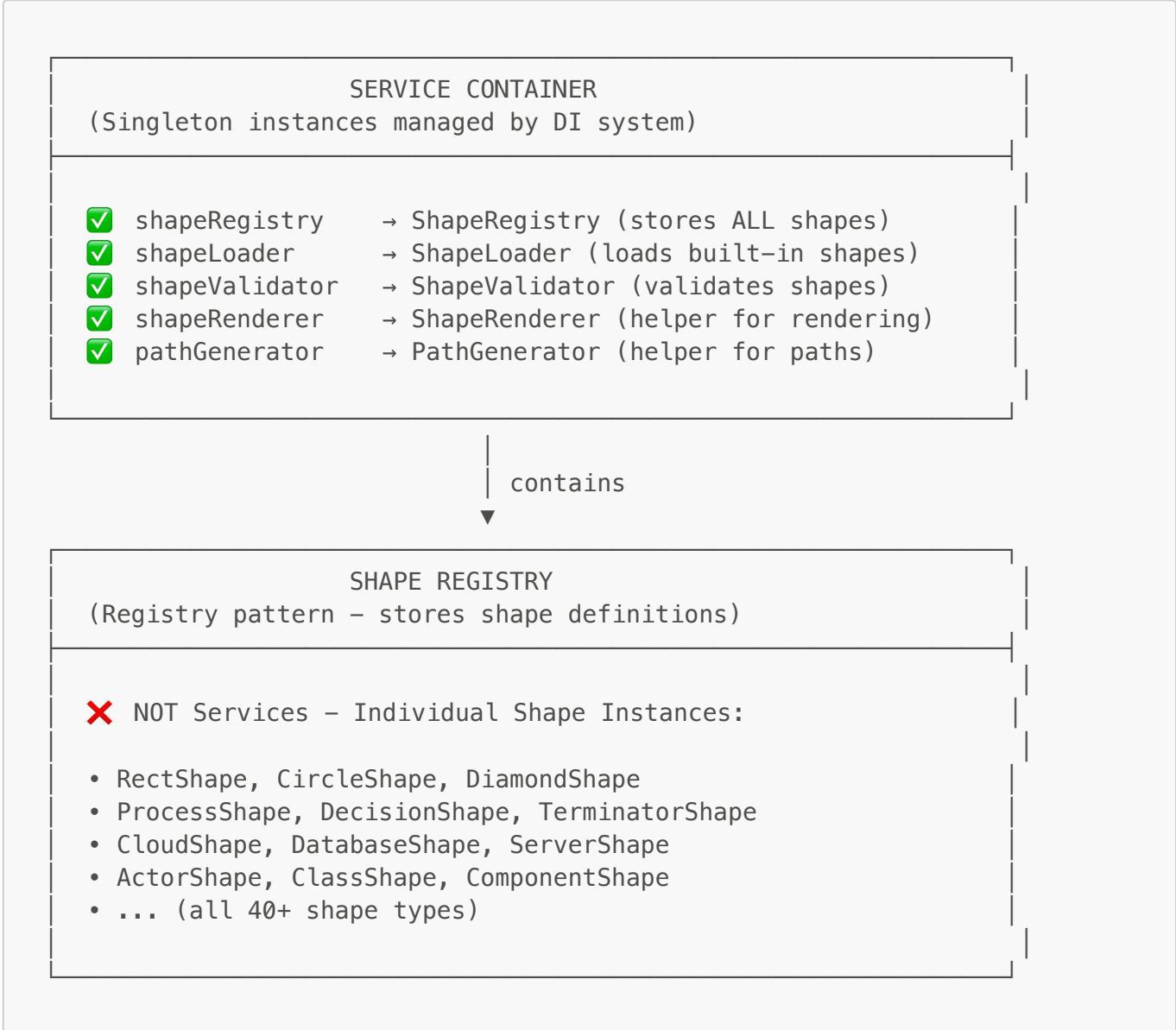
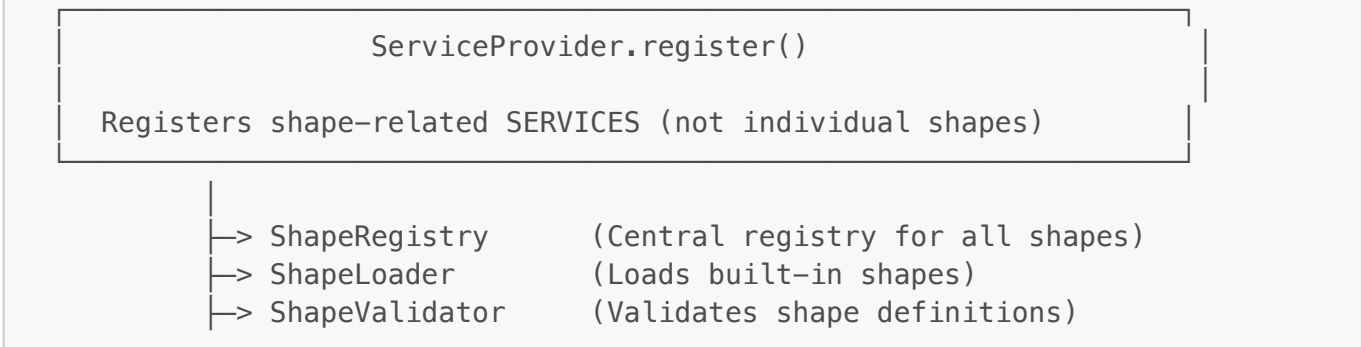
# SHAPE SYSTEM FLOWCHART - Complete Architecture

## 🎯 Overview: Shapes vs Services




## 📊 LAYER 1: Service Container Registration



```
          ├─> ShapeRenderer      (Helper: renders SVG elements)
          ├─> PathGenerator      (Helper: generates SVG paths)
          ├─> HandleManager      (Helper: manages resize handles)
          └─> PortManager        (Helper: manages connection ports)
```

---

## 📊 LAYER 2: Shape System Classes - Detailed Functions

### 1️⃣ ShapeRegistry (SERVICE)

**Location:** `src/shapes/registry/ShapeRegistry.js`

```
class ShapeRegistry {
  constructor() {
    this.shapes = new Map(); // Map<shapeType, BaseShape>
    this.categories = new Map(); // Map<category, Set<shapeType>>
    this.debugMode = this._isDebugMode();
    this._debug("ShapeRegistry initialized");
  }

  // FUNCTION: Register a shape definition
  register(shape) {
    this._debug(`Registering shape: ${shape.type}`);

    if (!this._validateShape(shape)) {
      console.error(`Invalid shape: ${shape.type}`);
      return false;
    }

    this.shapes.set(shape.type, shape);

    // Add to category
    if (!this.categories.has(shape.category)) {
      this.categories.set(shape.category, new Set());
    }
    this.categories.get(shape.category).add(shape.type);

    this._debug(`✓ Registered: ${shape.type} in ${shape.category}`);
    return true;
  }

  // FUNCTION: Get a specific shape
  getShape(type) {
    this._debug(`Getting shape: ${type}`);
    return this.shapes.get(type);
  }

  // FUNCTION: Get all shapes in a category
  getCategory(category) {
    this._debug(`Getting category: ${category}`);
    const shapeTypes = this.categories.get(category);
```

```
    if (!shapeTypes) return [];

    return Array.from(shapeTypes).map((type) => this.shapes.get(type));
  }

  // FUNCTION: Get all categories
  getAllCategories() {
    this._debug("Getting all categories");
    return Array.from(this.categories.keys());
  }

  // Debug helper
  _debug(message) {
    if (this.debugMode) {
      console.log(
        `%c[ShapeRegistry] ${message}`,
        "color: #FF6B6B; font-weight: bold"
      );
    }
  }

  _isDebugMode() {
    return (
      localStorage.getItem("debugMode") === "true" ||
      window.location.search.includes("debug=true")
    );
  }
}
```

**FUNCTION:** Central registry that stores and retrieves all shape definitions

---

## 2 ShapeLoader (SERVICE)

**Location:** `src/shapes/loader/ShapeLoader.js`

```
class ShapeLoader {
  constructor(shapeRegistry) {
    this.registry = shapeRegistry;
    this.debugMode = this._isDebugMode();
    this._debug("ShapeLoader initialized");
  }

  // FUNCTION: Load all built-in shapes
  static loadBuiltInShapes(registry) {
    const loader = new ShapeLoader(registry);
    loader._debug("Loading built-in shapes...");

    // Load basic shapes
    loader._loadBasicShapes();

    // Load flowchart shapes
```

```
    loader._loadFlowchartShapes();

    // Load network shapes
    loader._loadNetworkShapes();

    // Load UML shapes
    loader._loadUMLShapes();

    // Load arrow shapes
    loader._loadArrowShapes();

    // Load container shapes
    loader._loadContainerShapes();

    // Load text shapes
    loader._loadTextShapes();

    loader._debug(`✓ Loaded ${registry.shapes.size} shapes`);
  }

  // FUNCTION: Load basic shapes (rect, circle, diamond, etc.)
  _loadBasicShapes() {
    this._debug("Loading basic shapes...");

    import("./library/basic/rect/RectShape.js").then((module) => {
      this.registry.register(new module.RectShape());
      this._debug("  ✓ RectShape loaded");
    });

    import("./library/basic/circle/CircleShape.js").then((module) => {
      this.registry.register(new module.CircleShape());
      this._debug("  ✓ CircleShape loaded");
    });

    import("./library/basic/diamond/DiamondShape.js").then((module) => {
      this.registry.register(new module.DiamondShape());
      this._debug("  ✓ DiamondShape loaded");
    });

    // ... more shapes
  }

  // FUNCTION: Load flowchart shapes (process, decision, terminator, etc.)
  _loadFlowchartShapes() {
    this._debug("Loading flowchart shapes...");

    import("./library/flowchart/process/ProcessShape.js").then((module) =>
{
      this.registry.register(new module.ProcessShape());
      this._debug("  ✓ ProcessShape loaded");
    });

    import("./library/flowchart/decision/DecisionShape.js").then((module)
=> {
```

```
        this.registry.register(new module.DecisionShape());
        this._debug("  ✓ DecisionShape loaded");
      });

      // ... more flowchart shapes
    }

    // Debug helper
    _debug(message) {
      if (this.debugMode) {
        console.log(
          `%c[ShapeLoader] ${message}`,
          "color: #4ECDC4; font-weight: bold"
        );
      }
    }
  }
```

**FUNCTION:** Loads all built-in shape definitions into the registry

---

### 3️⃣ ShapeValidator (SERVICE)

**Location:** `src/shapes/loader/ShapeValidator.js`

```
class ShapeValidator {
  constructor() {
    this.debugMode = this._isDebugMode();
    this._debug("ShapeValidator initialized");
  }

  // FUNCTION: Validate shape definition
  validate(shape) {
    this._debug(`Validating shape: ${shape.type}`);

    const errors = [];

    // Check required fields
    if (!shape.type) errors.push("Missing type");
    if (!shape.category) errors.push("Missing category");
    if (!shape.render) errors.push("Missing render function");

    // Check dimensions
    if (shape.defaultWidth <= 0) errors.push("Invalid width");
    if (shape.defaultHeight <= 0) errors.push("Invalid height");

    // Check style
    if (!shape.defaultStyle) errors.push("Missing default style");

    if (errors.length > 0) {
      this._debug(`✗ Validation failed: ${errors.join(", ")}`);
      return { valid: false, errors };
```

```
    }

    this._debug(`✓ Validation passed`);
    return { valid: true, errors: [] };
  }

  // FUNCTION: Validate connection compatibility
  validateConnection(sourceShape, targetShape) {
    this._debug(
      `Validating connection: ${sourceShape.type} → ${targetShape.type}`
    );

    // Check if shapes have ports
    if (!sourceShape.defaultPorts || !targetShape.defaultPorts) {
      return false;
    }

    // Custom validation rules can go here
    return true;
  }

  _debug(message) {
    if (this.debugMode) {
      console.log(
        `%c[ShapeValidator] ${message}`,
        "color: #F7B731; font-weight: bold"
      );
    }
  }
}
```

**FUNCTION:** Validates shape definitions and connection rules

---

## 4️⃣ BaseShape (ABSTRACT CLASS - NOT A SERVICE)

**Location:** `src/shapes/base/BaseShape.js`

```
class BaseShape {
  constructor(config) {
    this.type = config.type;
    this.category = config.category;
    this.displayName = config.displayName;
    this.description = config.description;
    this.defaultWidth = config.defaultWidth || 120;
    this.defaultHeight = config.defaultHeight || 60;
    this.defaultPorts = config.defaultPorts || [];
    this.defaultStyle = config.defaultStyle || {};
    this.debugMode = this._isDebugMode();

    this._debug(`BaseShape created: ${this.type}`);
  }
```

```
  // FUNCTION: Render the shape as SVG
  render(x, y, width, height, style, label) {
    this._debug(`Rendering ${this.type} at (${x}, ${y})`);

    // Must be overridden by subclass
    throw new Error("render() must be implemented by subclass");
  }

  // FUNCTION: Create a shape instance from this definition
  createInstance(x, y, options = {}) {
    this._debug(`Creating instance of ${this.type}`);

    return {
      type: this.type,
      x: x,
      y: y,
      width: options.width || this.defaultWidth,
      height: options.height || this.defaultHeight,
      style: { ...this.defaultStyle, ...options.style },
      label: options.label || "",
      ports: [...this.defaultPorts],
    };
  }

  // FUNCTION: Get bounding box
  getBounds(x, y, width, height) {
    return {
      x: x,
      y: y,
      width: width,
      height: height,
    };
  }

  _debug(message) {
    if (this.debugMode) {
      console.log(
        `%c[${this.type}] ${message}`,
        "color: #A29BFE; font-weight: bold"
      );
    }
  }
}
```

**FUNCTION:** Abstract base class that all shapes extend from

---

## 5️⃣ ShapeBuilder (UTILITY - NOT A SERVICE)

**Location:** `src/shapes/base/ShapeBuilder.js`

```
class ShapeBuilder {
  constructor(type) {
    this.config = {
      type: type,
      category: "basic",
      displayName: type,
      description: "",
      defaultWidth: 120,
      defaultHeight: 60,
      defaultPorts: [],
      defaultStyle: {
        fill: "#ffffff",
        stroke: "#000000",
        strokeWidth: 2,
      },
    };
    this.debugMode = this._isDebugMode();
    this._debug(`ShapeBuilder initialized for: ${type}`);
  }

  // FUNCTION: Set category
  category(category) {
    this._debug(`Setting category: ${category}`);
    this.config.category = category;
    return this;
  }

  // FUNCTION: Set display name
  displayName(name) {
    this._debug(`Setting displayName: ${name}`);
    this.config.displayName = name;
    return this;
  }

  // FUNCTION: Set size
  size(width, height) {
    this._debug(`Setting size: ${width}x${height}`);
    this.config.defaultWidth = width;
    this.config.defaultHeight = height;
    return this;
  }

  // FUNCTION: Set style
  style(styleObj) {
    this._debug(`Setting style`);
    this.config.defaultStyle = { ...this.config.defaultStyle, ...styleObj
};
    return this;
  }

  // FUNCTION: Add ports
  ports(portsArray) {
    this._debug(`Adding ${portsArray.length} ports`);
```

```
    this.config.defaultPorts = portsArray;
    return this;
  }

  // FUNCTION: Build the final shape
  build() {
    this._debug(`Building shape: ${this.config.type}`);
    return new BaseShape(this.config);
  }

  _debug(message) {
    if (this.debugMode) {
      console.log(
        `%c[ShapeBuilder] ${message}`,
        "color: #FD79A8; font-weight: bold"
      );
    }
  }
}
```

**FUNCTION:** Fluent API for building complex shape definitions

---

### 6 ShapeRenderer (HELPER SERVICE)

**Location:** `src/shapes/helpers/ShapeRenderer.js`

```
class ShapeRenderer {
  constructor() {
    this.debugMode = this._isDebugMode();
    this._debug("ShapeRenderer initialized");
  }

  // FUNCTION: Render SVG element
  renderSVG(shape, x, y, width, height, style, label) {
    this._debug(`Rendering SVG for: ${shape.type}`);

    const g = document.createElementNS("http://www.w3.org/2000/svg", "g");
    g.setAttribute("transform", `translate(${x}, ${y})`);

    // Render shape-specific geometry
    const geometry = shape.render(0, 0, width, height, style, label);
    g.appendChild(geometry);

    // Render label if exists
    if (label) {
      const text = this._renderLabel(label, width, height);
      g.appendChild(text);
    }

    this._debug(`✓ Rendered: ${shape.type}`);
    return g;
```

```
  }

  // FUNCTION: Render text label
  _renderLabel(text, width, height) {
    const textEl = document.createElementNS(
      "http://www.w3.org/2000/svg",
      "text"
    );
    textEl.setAttribute("x", width / 2);
    textEl.setAttribute("y", height / 2);
    textEl.setAttribute("text-anchor", "middle");
    textEl.setAttribute("dominant-baseline", "middle");
    textEl.textContent = text;
    return textEl;
  }

  _debug(message) {
    if (this.debugMode) {
      console.log(
        `%c[ShapeRenderer] ${message}`,
        "color: #6C5CE7; font-weight: bold"
      );
    }
  }
}
```

**FUNCTION:** Helper service for rendering SVG elements

---

### 7 PathGenerator (HELPER SERVICE)

**Location:** `src/shapes/helpers/PathGenerator.js`

```
class PathGenerator {
  constructor() {
    this.debugMode = this._isDebugMode();
    this._debug("PathGenerator initialized");
  }

  // FUNCTION: Generate rectangle path
  rect(x, y, width, height, rx = 0) {
    this._debug(`Generating rect path: ${width}x${height}`);

    if (rx === 0) {
      return `M ${x} ${y} L ${x + width} ${y} L ${x + width} ${
        y + height
      } L ${x} ${y + height} Z`;
    }

    // Rounded rectangle
    return `M ${x + rx} ${y}
            L ${x + width - rx} ${y}
```

```
                Q ${x + width} ${y} ${x + width} ${y + rx}
                L ${x + width} ${y + height - rx}
                Q ${x + width} ${y + height} ${x + width - rx} ${y +
height}
                L ${x + rx} ${y + height}
                Q ${x} ${y + height} ${x} ${y + height - rx}
                L ${x} ${y + rx}
                Q ${x} ${y} ${x + rx} ${y} Z`;
  }

  // FUNCTION: Generate circle path
  circle(cx, cy, r) {
    this._debug(`Generating circle path: r=${r}`);

    return `M ${cx - r} ${cy}
                A ${r} ${r} 0 0 1 ${cx} ${cy - r}
                A ${r} ${r} 0 0 1 ${cx + r} ${cy}
                A ${r} ${r} 0 0 1 ${cx} ${cy + r}
                A ${r} ${r} 0 0 1 ${cx - r} ${cy} Z`;
  }

  // FUNCTION: Generate diamond path
  diamond(x, y, width, height) {
    this._debug(`Generating diamond path: ${width}x${height}`);

    const cx = x + width / 2;
    const cy = y + height / 2;

    return `M ${cx} ${y}
                L ${x + width} ${cy}
                L ${cx} ${y + height}
                L ${x} ${cy} Z`;
  }

  _debug(message) {
    if (this.debugMode) {
      console.log(
        `%c[PathGenerator] ${message}`,
        "color: #00B894; font-weight: bold"
      );
    }
  }
}
```

**FUNCTION:** Helper service for generating SVG path data

---

## 8️⃣ PortManager (HELPER SERVICE)

**Location:** `src/shapes/helpers/PortManager.js`

```
class PortManager {
  constructor() {
    this.debugMode = this._isDebugMode();
    this._debug("PortManager initialized");
  }

  // FUNCTION: Calculate port positions
  calculatePortPositions(x, y, width, height, ports) {
    this._debug(`Calculating ${ports.length} port positions`);

    return ports.map((port) => {
      let px, py;

      switch (port.side) {
        case "top":
          px = x + width * port.position;
          py = y;
          break;
        case "right":
          px = x + width;
          py = y + height * port.position;
          break;
        case "bottom":
          px = x + width * port.position;
          py = y + height;
          break;
        case "left":
          px = x;
          py = y + height * port.position;
          break;
      }

      this._debug(`  Port ${port.id}: (${px}, ${py})`);

      return {
        ...port,
        x: px,
        y: py,
      };
    });
  }

  // FUNCTION: Render port visualization
  renderPorts(ports) {
    this._debug(`Rendering ${ports.length} ports`);

    const g = document.createElementNS("http://www.w3.org/2000/svg", "g");
    g.classList.add("ports");

    ports.forEach((port) => {
      const circle = document.createElementNS(
        "http://www.w3.org/2000/svg",
        "circle"
```

```
      );
      circle.setAttribute("cx", port.x);
      circle.setAttribute("cy", port.y);
      circle.setAttribute("r", 4);
      circle.setAttribute("class", "port");
      circle.dataset.portId = port.id;

      g.appendChild(circle);
    });

    return g;
  }

  _debug(message) {
    if (this.debugMode) {
      console.log(
        `%c[PortManager] ${message}`,
        "color: #FDCB6E; font-weight: bold"
      );
    }
  }
}
```

**FUNCTION:** Helper service for managing connection ports

---

### 9 HandleManager (HELPER SERVICE)

**Location:** `src/shapes/helpers/HandleManager.js`

```
class HandleManager {
  constructor() {
    this.debugMode = this._isDebugMode();
    this._debug("HandleManager initialized");
  }

  // FUNCTION: Calculate resize handle positions
  calculateHandlePositions(x, y, width, height) {
    this._debug(`Calculating handle positions for ${width}x${height}`);

    return {
      nw: { x: x, y: y, cursor: "nw-resize" },
      n: { x: x + width / 2, y: y, cursor: "n-resize" },
      ne: { x: x + width, y: y, cursor: "ne-resize" },
      e: { x: x + width, y: y + height / 2, cursor: "e-resize" },
      se: { x: x + width, y: y + height, cursor: "se-resize" },
      s: { x: x + width / 2, y: y + height, cursor: "s-resize" },
      sw: { x: x, y: y + height, cursor: "sw-resize" },
      w: { x: x, y: y + height / 2, cursor: "w-resize" },
    };
  }
```

```
  // FUNCTION: Render resize handles
  renderHandles(handles) {
    this._debug("Rendering resize handles");

    const g = document.createElementNS("http://www.w3.org/2000/svg", "g");
    g.classList.add("handles");

    Object.entries(handles).forEach(([position, handle]) => {
      const rect = document.createElementNS(
        "http://www.w3.org/2000/svg",
        "rect"
      );
      rect.setAttribute("x", handle.x - 4);
      rect.setAttribute("y", handle.y - 4);
      rect.setAttribute("width", 8);
      rect.setAttribute("height", 8);
      rect.setAttribute("class", "handle");
      rect.setAttribute("data-position", position);
      rect.style.cursor = handle.cursor;

      g.appendChild(rect);
    });

    return g;
  }

  _debug(message) {
    if (this.debugMode) {
      console.log(
        `%c[HandleManager] ${message}`,
        "color: #E17055; font-weight: bold"
      );
    }
  }
}
```

**FUNCTION:** Helper service for managing resize handles

---

## 📊 COMPLETE DATA FLOW DIAGRAM

```
┌─────────────────────────────────────────────────────────────────────┐
│                          INITIALIZATION                             │
└─────────────────────────────────────────────────────────────────────┘

[1] main.js initializes
     │
     ├─> ServiceProvider.register(container)
     │    │
     │    ├─> Register ShapeRegistry
     │    │    └─> console: "[ShapeRegistry] ShapeRegistry initialized"
```

```
        │     │
        │     ├─> Register ShapeLoader
        │     │    └─> console: "[ShapeLoader] ShapeLoader initialized"
        │     │
        │     ├─> Register ShapeValidator
        │     │    └─> console: "[ShapeValidator] ShapeValidator initialized"
        │     │
        │     ├─> Register ShapeRenderer
        │     │    └─> console: "[ShapeRenderer] ShapeRenderer initialized"
        │     │
        │     ├─> Register PathGenerator
        │     │    └─> console: "[PathGenerator] PathGenerator initialized"
        │     │
        │     ├─> Register PortManager
        │     │    └─> console: "[PortManager] PortManager initialized"
        │     │
        │     └─> Register HandleManager
        │          └─> console: "[HandleManager] HandleManager initialized"
        │
        ├─> Get shapeRegistry from container
        │    └─> console: "[ServiceContainer] Getting service: shapeRegistry"
        │
        └─> ShapeLoader.loadBuiltInShapes(shapeRegistry)
             │
             └─> console: "[ShapeLoader] Loading built-in shapes..."
                  │
                  ├─> Load basic shapes
                  │    │
                  │    ├─> new RectShape()
                  │    │    └─> console: "[RectShape] BaseShape created:
rect"
                  │    │
                  │    ├─> registry.register(rectShape)
                  │    │    └─> console: "[ShapeRegistry] Registering shape:
rect"
                  │    │    └─> console: "[ShapeRegistry] ✓ Registered: rect
in basic"
                  │    │
                  │    ├─> new CircleShape()
                  │    │    └─> console: "[CircleShape] BaseShape created:
circle"
                  │    │
                  │    └─> registry.register(circleShape)
                  │         └─> console: "[ShapeRegistry] ✓ Registered:
circle in basic"
                  │
                  ├─> Load flowchart shapes
                  │    │
                  │    ├─> new ProcessShape()
                  │    │    └─> console: "[ProcessShape] BaseShape created:
process"
                  │    │
                  │    └─> registry.register(processShape)
                  │         └─> console: "[ShapeRegistry] ✓ Registered:
```

```
process in flowchart"
                │
                ├─> Load network shapes
                ├─> Load UML shapes
                ├─> Load arrow shapes
                ├─> Load container shapes
                └─> Load text shapes
                     │
                     └─> console: "[ShapeLoader] ✓ Loaded 45 shapes"

┌──────────────────────────────────────────────────────────────────────────┐
│                          RUNTIME USAGE                                   │
└──────────────────────────────────────────────────────────────────────────┘

[2] User clicks "Rectangle" in LeftPalette
     │
     ├─> LeftPalette.handleShapeClick('rect')
     │    └─> console: "[LeftPalette] Shape clicked: rect"
     │
     ├─> EventBus.emit('shape:selected', { type: 'rect' })
     │    └─> console: "[EventBus] Emitting: shape:selected"
     │
     └─> main.js receives 'shape:selected'
          │
          └─> StateManager.setMode('draw')
               └─> console: "[StateManager] Mode changed: select → draw"

[3] User clicks on canvas at (200, 150)
     │
     ├─> Canvas click handler
     │    └─> console: "[Canvas] Clicked at (200, 150)"
     │
     └─> NodeManager.createNode('rect', 200, 150)
          │
          ├─> console: "[NodeManager] Creating node: rect at (200, 150)"
          │
          ├─> shapeRegistry.getShape('rect')
          │    │
          │    ├─> console: "[ShapeRegistry] Getting shape: rect"
          │    └─> Returns RectShape instance
          │
          ├─> shape.createInstance(200, 150)
          │    │
          │    ├─> console: "[rect] Creating instance of rect"
          │    └─> Returns node data object
          │
          ├─> new NodeModel(nodeData)
          │    └─> console: "[NodeModel] Created: node_abc123"
          │
          ├─> new NodeView(model, shape, shapeRenderer)
          │    │
          │    ├─> console: "[NodeView] Creating view for: node_abc123"
          │    │
          │    ├─> shapeRenderer.renderSVG(shape, ...)
```

```
          |     |     |
          |     |     ├─> console: "[ShapeRenderer] Rendering SVG for: rect"
          |     |     |
          |     |     ├─> shape.render(...)
          |     |     |     |
          |     |     |     ├─> console: "[rect] Rendering rect at (200,
150)"
          |     |     |     |
          |     |     |     ├─> pathGenerator.rect(...)
          |     |     |     |     └─> console: "[PathGenerator] Generating
rect path: 120x60"
          |     |     |     |
          |     |     |     └─> Returns SVG path element
          |     |     |
          |     |     └─> console: "[ShapeRenderer] ✓ Rendered: rect"
          |     |
          |     ├─> portManager.calculatePortPositions(...)
          |     |     |
          |     |     ├─> console: "[PortManager] Calculating 4 port
positions"
          |     |     └─> console: "[PortManager]   Port top: (260, 150)"
          |     |
          |     ├─> portManager.renderPorts(ports)
          |     |     └─> console: "[PortManager] Rendering 4 ports"
          |     |
          |     └─> console: "[NodeView] ✓ View created for: node_abc123"
          |
          ├─> new NodeController(model, view)
          |     └─> console: "[NodeController] Controller created for:
node_abc123"
          |
          ├─> editor.addNodeElement(node.id, view.element)
          |     └─> console: "[Editor] Adding node to canvas: node_abc123"
          |
          ├─> EventBus.emit('node:created', node)
          |     └─> console: "[EventBus] Emitting: node:created"
          |
          └─> console: "[NodeManager] ✓ Node created: node_abc123"

[4] User selects node (to see resize handles)
     |
     ├─> SelectionManager.selectNode('node_abc123')
     |     └─> console: "[SelectionManager] Selected node: node_abc123"
     |
     └─> NodeView.showHandles()
           |
           ├─> handleManager.calculateHandlePositions(...)
           |     └─> console: "[HandleManager] Calculating handle positions
for 120x60"
           |
           └─> handleManager.renderHandles(handles)
                 └─> console: "[HandleManager] Rendering resize handles"
```

## 🎨 SHAPE CATEGORIES BREAKDOWN

```
┌──────────────────────────────────────────────────────────────────────┐
│                        SHAPE CATEGORIES                              │
└──────────────────────────────────────────────────────────────────────┘

1. BASIC (7 shapes)
   ├─ RectShape        → Rectangle
   ├─ CircleShape      → Circle
   ├─ EllipseShape     → Ellipse
   ├─ DiamondShape     → Diamond
   ├─ TriangleShape    → Triangle
   ├─ PolygonShape     → Polygon (configurable sides)
   └─ StarShape        → Star (configurable points)

2. FLOWCHART (7 shapes)
   ├─ ProcessShape           → Rectangle with label
   ├─ DecisionShape          → Diamond
   ├─ TerminatorShape        → Rounded rectangle
   ├─ DataShape              → Parallelogram
   ├─ DocumentShape          → Rectangle with wave bottom
   ├─ PredefinedProcessShape → Rectangle with double lines
   └─ PreparationShape       → Hexagon

3. NETWORK (7 shapes)
   ├─ ServerShape       → Server icon
   ├─ RouterShape       → Router icon
   ├─ CloudShape        → Cloud icon
   ├─ DatabaseShape     → Cylinder
   ├─ FirewallShape     → Firewall icon
   ├─ SwitchShape       → Switch icon
   └─ WorkstationShape  → Computer icon

4. UML (6 shapes)
   ├─ ClassShape       → Class box (3 sections)
   ├─ InterfaceShape   → Interface box
   ├─ ComponentShape   → Component icon
   ├─ ActorShape       → Stick figure
   ├─ NoteShape        → Folded note
   └─ PackageShape     → Package folder

5. ARROWS (3 shapes)
   ├─ StraightArrowShape → Straight arrow
   ├─ CurvedArrowShape   → Curved arrow
   └─ DoubleArrowShape   → Double-headed arrow

6. CONTAINERS (3 shapes)
   ├─ FrameShape     → Frame/border for grouping
   ├─ GroupShape     → Group container
   └─ SwimlaneShape  → Swimlane for process flows
```

```
7. TEXT (3 shapes)
   ├─ LabelShape   → Simple text label
   ├─ CalloutShape → Speech bubble
   └─ NoteTextShape→ Sticky note

TOTAL: ~45 built-in shapes
```

---

## 🔧 DEBUG MODE ACTIVATION

Add debug logging to every class:

```
// Method 1: URL parameter
//localhost:8000/?debug=true

// Method 2: localStorage
http: localStorage.setItem("debugMode", "true");
location.reload();

// Method 3: Console command
window.enableDebug = () => {
  localStorage.setItem("debugMode", "true");
  location.reload();
};

window.disableDebug = () => {
  localStorage.removeItem("debugMode");
  location.reload();
};

// Usage
enableDebug();
```

Each class checks for debug mode:

```
_isDebugMode() {
    return localStorage.getItem('debugMode') === 'true' ||
           window.location.search.includes('debug=true');
}
```

---

## 📝 SUMMARY

| Class | Type | Registered In | Function |
|---|---|---|---|
| **ShapeRegistry** | SERVICE | ServiceContainer | Central registry for all shape definitions |

| Class | Type | Registered In | Function |
|---|---|---|---|
| **ShapeLoader** | SERVICE | ServiceContainer | Loads built-in shapes into registry |
| **ShapeValidator** | SERVICE | ServiceContainer | Validates shape definitions |
| **ShapeRenderer** | HELPER | ServiceContainer | Renders SVG elements |
| **PathGenerator** | HELPER | ServiceContainer | Generates SVG path data |
| **PortManager** | HELPER | ServiceContainer | Manages connection ports |
| **HandleManager** | HELPER | ServiceContainer | Manages resize handles |
| **BaseShape** | ABSTRACT CLASS | ShapeRegistry (instances) | Base class for all shapes |
| **ShapeBuilder** | UTILITY | N/A (used directly) | Fluent API for building shapes |
| **RectShape, CircleShape, etc.** | SHAPE INSTANCE | ShapeRegistry | Individual shape definitions |

**Key Insight:**

- Services (7 classes) → Registered in **ServiceContainer**
- Shapes (45+ instances) → Registered in **ShapeRegistry**
- ShapeRegistry itself is a SERVICE that holds all shapes